PROJECT REPORT PRESENTATION

LEX & YACC / BISON IMPLEMENTATION

Conditional Statement Compiler

A comprehensive front-end compiler pipeline covering lexical scanning, context-free parsing, Abstract Syntax Tree construction, type verification, and web GUI execution.

SUBMITTED BY

Abhinav A (7) • Abiya Susan Cherian (14) • Akshay Wilson (26)
Ambadi B (44) • Angelina Anna Antony (52) • Ashmy Philo Joji (67)

01. INTRODUCTION

Background & Problem Statement

Background

Conditional control structures (`if`, `if-else`, nested blocks) form the backbone of modern imperative execution paths.

A robust compiler front end must tokenize raw streams, validate exact structural productions, construct abstract syntax representations, and enforce static semantic constraints.

Problem Statement

Designing a synchronized parser requires managing lexical token streams, resolving grammar ambiguities like dangling-else, and detecting syntax violations.

This project resolves these challenges using Flex and YACC/Bison coupled with a dynamic AST builder and a custom type-checking symbol manager.

02. ARCHITECTURE

Compiler Pipeline & Execution Flow

1
Source Input
Flask Web GUI / .src file

→

2
Flex Lexer
Tokenization (lexer.l)

→

3
Bison Parser
Syntax Check (parser.y)

→

4
AST & Semantics
Tree & Symbol Table

Execution Flow: Flask web interface writes editor input to a temporary source file, invokes the compiled binary executable `conditional_compiler`, captures stdout/stderr via HTML/CSS/JS frontend, and renders the AST and semantic state.

03. LEXICAL ANALYSIS

Tokenization Rules (lexer.l)

Keywords & Identifiers

Matches `if` and `else` reserved words. Identifier string tokens are dynamically duplicated into allocated memory for yylval.

Literals & Operators

Converts numeric literals via `atof()` into `yylval.number`. Scans arithmetic, relational (<, ⩾), and logical operators.

Delimiters & Diagnostics

Parses grouping symbols, braces, and semicolons. Increments line tracking counters and returns `INVALID` on illegal characters.

04. SYNTAX SPECIFICATION

Context-Free Grammar Rules

```
program       → statement_list
statement_list → statement_list statement | statement
statement     → assignment | if_statement | block
if_statement  → IF '(' condition ')' statement
              | IF '(' condition ')' statement ELSE statement
block         → '{' statement_list '}'
assignment    → IDENTIFIER '=' expression ';'
condition     → expression
```

Precedence & Ambiguity Control

The grammar enforces operator precedence cascades (arithmetic < relational < equality < logical AND < logical OR).

Dangling-else ambiguity is explicitly resolved in YACC/Bison by giving the `ELSE` keyword higher precedence over standard single `IF` evaluation.

05. PARSER DESIGN

Syntax Analyzer Implementation (parser.y)

Driver Invocation

The C driver executes `yyparse()`. A non-zero return code or NULL AST root signals syntax error state.

Block & Scope Structure

Enforces strict matching for nested statement blocks enclosed by opening and closing braces.

Diagnostic Output

Reports exact token failure locations, missing semicolons, or unclosed parentheses during compilation.

06. AST ARCHITECTURE

Abstract Syntax Tree (ast.c / ast.h)

Node Data Structures

Nodes capture operations, literal values, variable symbols, and sub-trees. Each node maintains a `next` pointer to chain statements within execution blocks.

Helper API: Functions construct nodes, append statement lists, recursively print indented AST trees, and safely release allocated heap pointers.

Node Type	Pointers / Children
NODE_IF	Condition sub-tree + Then statement block
NODE_IF_ELSE	Condition + Then block + Else block
NODE_ASSIGN	Target Identifier string + Expression sub-tree
NODE_BIN_OP	Operator character/string + Left & Right children

07. SEMANTIC ANALYSIS

Static Type Checking & Symbol Table

Enforced Semantic Verification Rules

- **Condition Type Verification:** Ensures `if` condition evaluates strictly to Boolean.
- **Logical & Relational Verification:** Checks that logical operators receive Boolean inputs, and relational operators evaluate numeric inputs.
- **Division By Zero Prevention:** Scans numeric literal expressions to intercept explicit `/ 0` operations.
- **Type Match:** Asserts target variable type matches incoming evaluated expression type.

Driver Pre-registered Table

`x`	NUMERIC
`y`	NUMERIC
`z`	NUMERIC
`flag`	BOOLEAN

08. TESTING

Testing Strategy & Validation Matrix

Case	Input Source Snippet	Target Validation Objective	Expected Outcome
V1	`if (x > 5) { y = 20; }`	Valid single IF condition & assignment	**Passes Lex, Syntax & Semantics**
V2	`if (x > 5) { y = 20; } else { y = 1; }`	Valid IF-ELSE branching execution	**Passes Lex, Syntax & Semantics**
V3	`if (x > 5) { if (y < 30) { z = 2; } }`	Valid nested conditional hierarchy	**Passes Lex, Syntax & Semantics**
I1	`if (x > 5 { y = 1; }`	Syntax check: missing closing parenthesis	**Syntax Error (Parser Failure)**
I5	`if (x + 1) { y = 2; }`	Semantic check: numeric condition expression	**Semantic Error (Type Mismatch)**

09. EVALUATION

Current Project Limitations

No Runtime Declarations

Lacks grammar rules for inline type declarations (`int x;`). Relies exclusively on static pre-registration.

Fixed Symbol Table

Symbols are fixed to pre-registered identifiers (`x, y, z, flag`). Unregistered variables trigger errors.

Web Interface Dependency

The Flask web GUI requires a pre-built binary compiled executable (`conditional_compiler`) at path.

10. TEAM RESPONSIBILITIES

Individual Contributions (Appendix A)

MEMBER 1 (ROLL 44)

Ambadi B

Lexical Scanner Development (lexer.l)

MEMBER 2 (ROLL 52)

Angelina Anna Antony

Context-Free Grammar Specs & Docs

MEMBER 3 (ROLL 67)

Ashmy Philo Joji

YACC/Bison Syntax Parser (parser.y)

MEMBER 4 (ROLL 07)

Abhinav A

AST Infrastructure & Semantic Analysis

MEMBER 5 (ROLL 26)

Akshay Wilson

Makefile, Main Driver & Testing

MEMBER 6 (ROLL 14)

Abiya Susan Cherian

Project Documentation & Presentation

CONCLUSION

Summary & Future Work

Demonstrates end-to-end front-end processing for conditional constructs. Future extensions include dynamic inline variable declarations, improved error recovery, source-line location tracking, and IR generation.

Thank You — Questions & Answers